

Skills und Prompts für Arbeitspakete und Reviews

22.07.2026 · adesso

Exported by Michael Orologas

Kurzantwort

Ja – aber es gibt meines Wissens keine einzelne, etablierte Sammlung, die exakt deinen kompletten Ablauf „Git-Issue → Review → Verbesserung → erneutes Review → Stop bei Qualitätsschwelle“ abdeckt.

Die passenden Bausteine liegen aktuell in drei Bereichen:

- 1. **Spec-Driven Development** für die Erstellung und Zerlegung von Arbeitspaketen
- 2. **Agent Skills** für wiederverwendbare Workflows und feste Qualitätsregeln
- 3. **LLM-Evals/Rubrics** für reproduzierbare Reviews und Regressionstests

Für deinen n8n-Workflow würde ich nicht nach einem einzigen Mega-Prompt suchen, sondern diese Bausteine kombinieren.

Die besten Anlaufstellen

Ressource	Eignung für deinen Fall
GitHub Spec Kit	Wahrscheinlich der beste Ausgangspunkt. Es arbeitet mit den Phasen <code>specify</code> → <code>plan</code> → <code>tasks</code> → <code>implement</code> und bietet zusätzlich <code>analyze</code> , <code>checklist</code> , <code>converge</code> sowie <code>taskstoissues</code> . Besonders interessant: Vorlagen, Presets und Bundles lassen sich an eigene Organisationsstandards anpassen.
Addy Osmani – Agent Skills	Sehr gute Sammlung für strukturierte Engineering-Workflows. Relevant sind insbesondere <code>spec-driven-development</code> , <code>planning-and-task-breakdown</code> , <code>code-review-and-quality</code> , <code>context-engineering</code> und <code>doubt-driven-development</code> . Die Skills enthalten Prozesse, Prüfschritte und Exit-Kriterien statt nur allgemeiner Prompt-Texte.
Anthropic Agent Skills	Gute Referenz für den grundsätzlichen Aufbau eines Skills mit <code>SKILL.md</code> , Metadaten, Anweisungen, Beispielen und optionalen Ressourcen/Skripten. Die Sammlung ist eher Referenz und Ausgangspunkt als fertige Lösung für deinen konkreten Use Case.
VoltAgent – Awesome Agent Skills	Große, kuratierte Sammlung mit Skills aus offiziellen Entwicklerteams und der Community. Dort findest du unter

	anderem Skills für GitHub, Issue-Erstellung, Review, Testing und Requirements Engineering. Vor der Übernahme solltest du jeden Skill fachlich und sicherheitstechnisch prüfen.
Promptfoo	Für deinen Fall sehr wichtig: Prompts und Reviews testen, mehrere Prompt-/Modellvarianten vergleichen, Regressionstests ausführen und Qualitätsgates in CI/CD integrieren. Das ist eher die Testinfrastruktur für deinen Workflow als eine Prompt-Sammlung.
Prompt Engineering Guide	Gute Referenz für Prompt Chaining, Evaluation, strukturierte Ausgabe, Agenten und Review-Techniken. Weniger spezialisiert auf Arbeitspakete, aber nützlich zum Verständnis der Muster.
GitHub Issue Forms	Für die Eingabeseite kannst du mit GitHub Issue Forms Pflichtfelder, Dropdowns, Validierungen, Labels und Standardwerte definieren. Die Antworten werden anschließend als Markdown im Issue abgelegt.

Die Quellen sind unterschiedlich ausgereift: Spec Kit und Addy Osmani sind für deinen konkreten Workflow am interessantesten; Anthropic und VoltAgent liefern die Skill-Struktur; Promptfoo und Anthropic Evals liefern die Qualitätssicherung. [\[1\]](#) [\[2\]](#) [\[3\]](#) [\[4\]](#) [\[5\]](#) [\[6\]](#)

Meine Empfehlung für deinen n8n-Workflow

Ich würde die Lösung in **vier getrennte Artefakte** aufteilen:

```
work-package-skill.md
review-rubric.yaml
review-output-schema.json
improver-prompt.md
```

1. **work-package-skill.md**

Enthält die Regeln für die Erstellung und Überarbeitung eines Arbeitspakets:

```
# Work Package Skill

## Ziel
Erzeuge ein umsetzbares Arbeitspaket für ein Git Issue.

## Grundregeln
- Keine Annahmen als Fakten darstellen.
- Fehlende Informationen als offene Frage markieren.
- Ein Arbeitspaket beschreibt ein klares Ergebnis.
- Scope, Nicht-Scope und Abnahmekriterien müssen getrennt sein.
```

- Abnahmekriterien müssen prüfbar sein.
- Keine technischen Details erfinden, wenn sie nicht aus dem Kontext hervorgehen.
- Bestehende gültige Inhalte bei einer Überarbeitung erhalten.

Pflichtabschnitte

1. Ziel und Nutzen
2. Kontext
3. Scope
4. Nicht im Scope
5. Ergebnis / Deliverables
6. Abnahmekriterien
7. Abhängigkeiten
8. Annahmen
9. Risiken
10. Offene Fragen
11. Definition of Done

Wichtig: Ein Skill sollte kein „Bitte schreibe ein gutes Arbeitspaket“ sein, sondern ein **Prozess mit Regeln, Prüfschritten und Abbruchkriterien**. Genau diesen Unterschied machen beispielsweise die Skills von Addy Osmani: Sie trennen Definition, Planung, Umsetzung, Prüfung und Review. [\[cite?c5.2\]](#)

2. review-rubric.yaml

Die Rubrik sollte nicht nur ein Gesamtscore-Feld enthalten. Besser sind einzelne Kriterien mit festen Bewertungsankern.

Beispiel:

```
rubric_version: "1.0"

criteria:
  - id: objective
    name: "Ziel und Nutzen"
    weight: 15
    scale:
      0: "Kein klares Ziel erkennbar"
      1: "Ziel vorhanden, aber unpräzise oder ohne nachvollziehbaren Nutzen"
      2: "Ziel und erwarteter Nutzen sind klar beschrieben"

  - id: scope
    name: "Scope und Abgrenzung"
    weight: 15
    scale:
      0: "Scope fehlt oder ist widersprüchlich"
      1: "Scope teilweise erkennbar"
      2: "In Scope und Out of Scope sind klar abgegrenzt"

  - id: deliverables
    name: "Ergebnis und Deliverables"
    weight: 15
```

scale:

0: "Kein konkretes Ergebnis"

1: "Ergebnis grob beschrieben"

2: "Ergebnis ist konkret und überprüfbar"

- id: acceptance_criteria

name: "Abnahmekriterien"

weight: 25

scale:

0: "Keine Abnahmekriterien"

1: "Abnahmekriterien vorhanden, aber teilweise nicht prüfbar"

2: "Alle wesentlichen Kriterien sind eindeutig und prüfbar"

- id: dependencies

name: "Abhängigkeiten und Annahmen"

weight: 10

scale:

0: "Relevante Abhängigkeiten fehlen"

1: "Teilweise dokumentiert"

2: "Relevante Abhängigkeiten und Annahmen sind nachvollziehbar"

- id: risks

name: "Risiken und offene Fragen"

weight: 10

scale:

0: "Keine Risiken oder offenen Fragen trotz erkennbarer Unsicherheit"

1: "Teilweise dokumentiert"

2: "Relevante Risiken und offene Fragen sind konkret benannt"

- id: size

name: "Größe und Umsetzbarkeit"

weight: 10

scale:

0: "Zu groß, zu unscharf oder nicht realistisch umsetzbar"

1: "Grundsätzlich umsetzbar, aber weitere Zerlegung sinnvoll"

2: "In einem angemessenen Arbeitsumfang umsetzbar"

hard_gates:

- id: has_objective

description: "Ein konkretes Ziel muss vorhanden sein"

- id: has_acceptance_criteria

description: "Mindestens ein prüfbares Abnahmekriterium muss vorhanden sein"

- id: no_blocking_open_question

description: "Keine unbeantwortete Frage darf die Umsetzung vollständig blockieren"

- id: valid_structure

description: "Alle Pflichtabschnitte müssen vorhanden sein"

threshold:

minimum_score: 80

maximum_iterations: 4

Für Arbeitspakete sind **INVEST** und **prüfbare Acceptance Criteria** gute Inspirationsquellen. INVEST steht für Independent, Negotiable, Valuable, Estimable, Small und Testable. ^[15] Acceptance Criteria sollten klar, unabhängig, messbar und objektiv prüfbar sein. ^[20] Ich würde INVEST aber nicht unverändert übernehmen: Für technische oder interne Arbeitspakete sind zusätzlich **Scope, Abhängigkeiten, Risiken und technische Randbedingungen** notwendig.

3. review-output-schema.json

Der Reviewer sollte **nicht direkt den Markdown-Kommentar erzeugen**, sondern zuerst maschinenlesbares JSON zurückgeben:

```
{
  "schema_version": "1.0",
  "decision": "REVISE",
  "criteria": [
    {
      "id": "objective",
      "score": 2,
      "max_score": 2,
      "status": "PASS",
      "evidence": "Das Ziel beschreibt die Bereitstellung eines automatisierten Review-Prozesses.",
      "finding": null
    },
    {
      "id": "acceptance_criteria",
      "score": 1,
      "max_score": 2,
      "status": "REVISE",
      "evidence": "Abnahmekriterien sind vorhanden, enthalten aber keine konkreten Prüfergebnisse.",
      "finding": "Es fehlt eine eindeutige Aussage, wie die erfolgreiche Umsetzung verifiziert wird."
    }
  ],
  "hard_gates": [
    {
      "id": "has_objective",
      "passed": true
    },
    {
      "id": "has_acceptance_criteria",
      "passed": false,
      "reason": "Mindestens ein Kriterium ist nicht objektiv prüfbar."
    }
  ],
  "findings": [
```

```
{
  "id": "AC-001",
  "severity": "MAJOR",
  "section": "Abnahmekriterien",
  "problem": "Das Kriterium ist zu allgemein.",
  "required_change": "Ein messbares Prüfergebnis ergänzen."
}
],
"next_action": "IMPROVE"
}
```

Den **Gesamtscore sollte n8n selbst berechnen**. Der LLM-Reviewer liefert nur die Einzelscores und Begründungen. So verhinderst du, dass das Modell beispielsweise behauptet, der Score sei 87, obwohl die Einzelwerte nur 72 ergeben.

4. improver-prompt.md

Der Improver sollte nicht das ganze Issue frei neu schreiben. Er sollte nur auf die Findings reagieren:

Improver Instructions

Überarbeite das Arbeitspaket ausschließlich anhand der übergebenen Review-Findings.

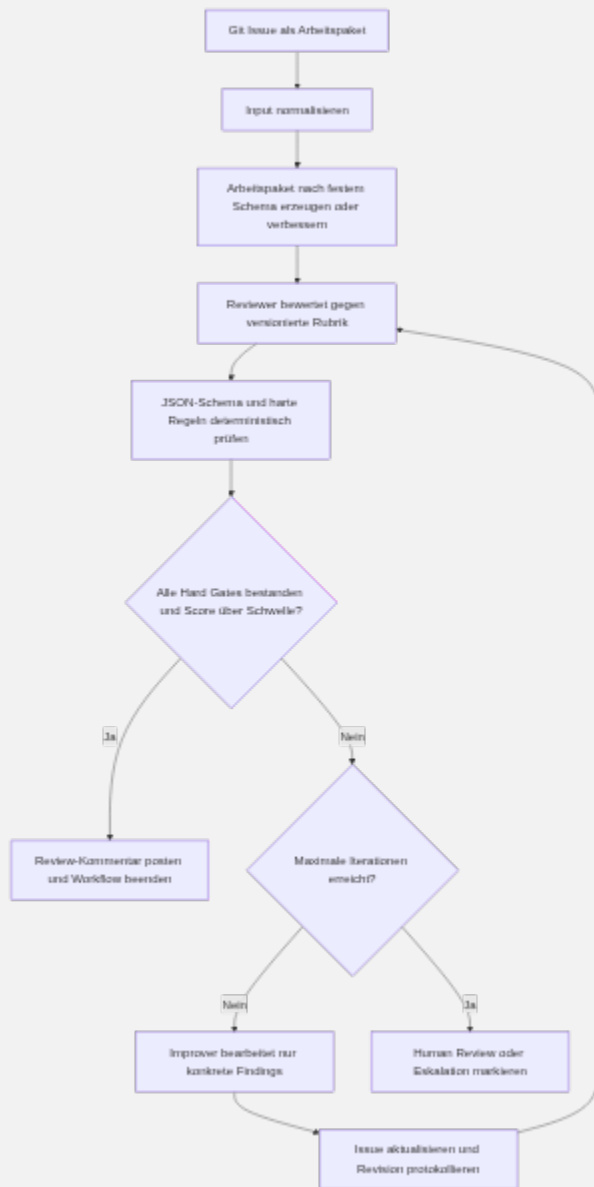
Regeln:

1. Akzeptierte Inhalte dürfen nicht ohne Grund verändert werden.
2. Nur Findings mit Severity BLOCKER oder MAJOR müssen zwingend behoben werden.
3. MINOR-Findings dürfen verbessert werden, wenn sie keine neuen Annahmen erzeugen.
4. Fehlende Informationen niemals erfinden.
5. Wenn eine Information erforderlich ist, aber nicht vorliegt:
 - als offene Frage aufnehmen
 - nicht selbst entscheiden
6. Die festgelegten Markdown-Überschriften und die Reihenfolge beibehalten.
7. Gib nur das vollständige aktualisierte Arbeitspaket zurück.
8. Gib keinen Review-Kommentar und keine Bewertung zurück.

Damit sind die Rollen sauber getrennt:

- **Generator:** erstellt das Arbeitspaket
 - **Reviewer:** bewertet nur
 - **Improver:** behebt Findings
 - **n8n-Code:** berechnet Score, prüft Schema und entscheidet über den Loop
-

Empfohlener Ablauf



Was n8n deterministisch erledigen sollte

Nicht alles sollte vom LLM kommen. In n8n würde ich diese Prüfungen mit Code oder normalen Nodes erledigen:

- Ist das Ergebnis gültiges JSON?
- Sind alle Pflichtfelder vorhanden?
- Sind alle Pflichtüberschriften im Issue enthalten?
- Ist der Score korrekt aus den Einzelscores berechnet?
- Sind alle Hard Gates erfüllt?
- Ist die maximale Iterationszahl erreicht?
- Hat sich der Issue-Inhalt tatsächlich verändert?
- Wurde derselbe Finding mehrfach erfolglos bearbeitet?

- Ist der Output zu lang?
- Gibt es verbotene oder nicht erlaubte Felder?
- Stimmen `rubric_version`, `prompt_version` und `model_version`?

Das passt zu den aktuellen Empfehlungen für Agent-Evaluationen: Code-basierte Grader sollten für objektive Kriterien verwendet werden, model-basierte Grader für offene oder qualitative Kriterien und menschliche Prüfung zur Kalibrierung. [?cite?c14.1?](#)

Nicht nur nach einem Score stoppen

Ich würde nicht ausschließlich mit folgender Bedingung arbeiten:

```
score > 80
```

Besser:

```
PASS =
weighted_score >= 80
AND all_hard_gates_passed
AND no_blocker_findings
AND no_major_finding_without_evidence
```

Ein Arbeitspaket kann beispielsweise einen hohen Score erreichen, obwohl ein kritisches Abnahmekriterium fehlt. Deshalb sollten **Hard Gates und Score getrennt** behandelt werden.

Außerdem sollte es immer eine Begrenzung geben:

```
max_iterations = 4
```

Nach vier erfolglosen Iterationen:

```
status = HUMAN_REVIEW_REQUIRED
```

Sonst kann sich der Workflow in semantischen Kleinständerungen oder einem Score-Pingpong verlieren.

So würde ich die Review-Kommentare formatieren

Der Kommentar kann anschließend vollständig deterministisch aus dem JSON erzeugt werden:

```
## 🤖 Arbeitspaket-Review

- **Revision:** 3
- **Entscheidung:** REVISE
- **Score:** 72/100
- **Schwelle:** 80/100
- **Rubrik:** 1.0
```

```
### Ergebnis
```

Das Arbeitspaket ist grundsätzlich verständlich, aber die Abnahmekriterien sind noch nicht ausreichend prüfbar.

Kriterien

Kriterium	Score	Status
---	---	---
Ziel und Nutzen	2/2	✓
Scope und Abgrenzung	2/2	✓
Ergebnis / Deliverables	2/2	✓
Abnahmekriterien	1/2	⚠
Abhängigkeiten	1/2	⚠

Findings

- **MAJOR – AC-001:** Das Abnahmekriterium ist nicht objektiv prüfbar.
- **MINOR – RISK-001:** Ein erkennbares Betriebsrisiko ist nicht dokumentiert.

Nächster Schritt

Das Arbeitspaket wird anhand der Findings überarbeitet.

Im Issue kannst du zusätzlich unsichtbare Metadaten speichern:

```
<!-- ai-work-package-meta
{
  "revision": 3,
  "rubric_version": "1.0",
  "prompt_version": "2026-07-01",
  "status": "REVISE",
  "score": 72,
  "workflow_run_id": "..."
}
-->
```

So kann n8n zuverlässig erkennen, welche Revision zuletzt bewertet wurde.

Für die Konsistenz besonders wichtig

1. Prompt, Skill und Rubrik versionieren

Nicht nur den Prompt speichern, sondern gemeinsam:

```
prompt_version
skill_version
rubric_version
model_version
workflow_version
```

2. Review und Verbesserung in getrennten LLM-Aufrufen

Kein Prompt sollte gleichzeitig sagen:

„Bewerte das Arbeitspaket, verbessere es und entscheide anschließend, ob es gut genug ist.“

Das begünstigt Selbstbestätigung. Reviewer und Improver sollten getrennte

Aufrufe mit getrennten Rollen und möglichst frischem Kontext sein.

3. Beispiele verwenden

Lege mindestens folgende Beispiele ab:

```
examples/  
excellent-work-package.md  
weak-work-package.md  
ambiguous-acceptance-criteria.md  
over-scoped-work-package.md  
missing-dependencies.md
```

Der Reviewer sollte pro Kriterium nicht nur eine Beschreibung, sondern auch **Pass- und Fail-Beispiele** kennen.

4. Golden Set aufbauen

Starte mit ungefähr 20–50 echten, anonymisierten Arbeitspaketen:

```
{  
  "input_issue": "...",  
  "expected_findings": [  
    "missing_acceptance_criteria",  
    "scope_too_large"  
  ],  
  "expected_decision": "REVISE"  
}
```

Diese Fälle kannst du bei jeder Änderung von Prompt, Rubrik oder Modell erneut prüfen. Promptfoo ist dafür eine passende Infrastruktur, weil es Prompts und Modelle vergleichen sowie automatische Evaluations- und CI/CD-Prüfungen ausführen kann. [?cite?c5.7?](#)

Meine konkrete Empfehlung

Für einen ersten belastbaren Prototypen würde ich Folgendes verwenden:

4. **GitHub Spec Kit als methodische Vorlage**
 - specify
 - plan
 - tasks
 - analyze
 - checklist
 - converge
2. **Addy Osmani Agent Skills als Skill-Struktur**
 - Prozess
 - Qualitätsgates
 - Verification
 - Anti-Rationalization-Regeln
3. **GitHub Issue Form oder eigenes Markdown-Schema**
 - Pflichtfelder

- Scope
- Abnahmekriterien
- Abhängigkeiten
- Risiken
- 4. **Eigene versionierte Review-Rubrik**
 - 6–8 Kriterien
 - pro Kriterium 0–2 Punkte
 - Hard Gates
 - klare Pass-/Fail-Beispiele
- 5. **n8n für die Orchestrierung**
 - LLM-Aufrufe
 - JSON-Parsing
 - Score-Berechnung
 - GitHub-Updates
 - Loop-Counter
 - Abbruchlogik
- 6. **Promptfoo oder ein kleines eigenes Testskript**
 - Golden Set
 - Regressionstests
 - Vergleich neuer Rubrik-/Prompt-Versionen

Der entscheidende Punkt ist: **Nicht das Modell soll die Qualitätsschleife kontrollieren, sondern n8n**. Das Modell liefert Bewertung und Änderungsvorschläge; n8n entscheidet deterministisch, ob weiter iteriert, beendet oder eskaliert wird. Anthropic beschreibt für Agent-Evaluationen ebenfalls die Kombination aus Tasks, Trials, unterschiedlichen Grader-Typen und Regressionsevaluationen als robusteren Ansatz gegenüber einer einzelnen freien Bewertung. [\[cite?c14.1?\]](#)